LAB 7

MST Implementation using Kruskal's and Prim's Algorithm

Algorithm – Kruskal's Algorithm

1. Start with all vertices as separate sets.
2. Sort all edges in non-decreasing order of their weights.
3. Select the smallest-weight edge.
4. Check whether adding the edge creates a cycle using Disjoint Set Union (DSU).
5. If it does not create a cycle, add the edge to the MST.
6. Repeat until the MST contains V - 1 edges.
7. Calculate and display the total weight of the MST.

Algorithm – Prim's Algorithm

1. Select any vertex as the starting vertex.
2. Mark the starting vertex as included in the MST.
3. Among all edges connecting an included vertex to an unvisited vertex, select the edge with minimum weight.
4. Add the selected edge and the new vertex to the MST.
5. Update the minimum edge (key) values for adjacent unvisited vertices.
6. Repeat until all vertices are included.
7. Calculate and display the total weight of the MST.

C++ Program

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

struct Edge {
int u, v, w;
};

bool compareEdge(Edge a, Edge b) {
return a.w < b.w;
}

// Disjoint Set Union for Kruskal's Algorithm
class DSU {
vector<int> parent, rankv;

public:
DSU(int n) {
parent.resize(n);
rankv.assign(n, 0);
for (int i = 0; i < n; i++)
parent[i] = i;
}

int find(int x) {
if (parent[x] == x)
return x;
return parent[x] = find(parent[x]);
}

bool unite(int a, int b) {
a = find(a);
b = find(b);

if (a == b)
return false;

if (rankv[a] < rankv[b])
swap(a, b);

parent[b] = a;
if (rankv[a] == rankv[b])
rankv[a]++;

return true;
}
};

// Kruskal's Algorithm
void kruskal(int n, vector<Edge> edges) {
sort(edges.begin(), edges.end(), compareEdge);

DSU dsu(n);
int total = 0, count = 0;

cout << "\nKruskal's MST:\n";
for (auto e : edges) {
if (dsu.unite(e.u, e.v)) {
cout << e.u << " - " << e.v << " : " << e.w << endl;
total += e.w;
count++;

if (count == n - 1)
break;
}
}

cout << "Total MST Weight = " << total << endl;
}

// Prim's Algorithm
void prim(int n, const vector<vector<pair<int, int>>>& graph) {
const int INF = 1000000000;
vector<int> key(n, INF), parent(n, -1);
vector<bool> used(n, false);

key[0] = 0;
int total = 0;

cout << "\nPrim's MST:\n";

for (int i = 0; i < n; i++) {
int u = -1;

for (int j = 0; j < n; j++) {
if (!used[j] && (u == -1 || key[j] < key[u]))
u = j;
}
```

```
used[u] = true;

if (parent[u] != -1) {
cout << parent[u] << " - " << u << " : " << key[u] << endl;
total += key[u];
}

for (auto edge : graph[u]) {
int v = edge.first;
int w = edge.second;

if (!used[v] && w < key[v]) {
key[v] = w;
parent[v] = u;
}
}
}

cout << "Total MST Weight = " << total << endl;
}

int main() {
int n = 5;

vector<Edge> edges = {
{0, 1, 2},
{0, 3, 6},
{1, 2, 3},
{1, 3, 8},
{1, 4, 5},
{2, 4, 7},
{3, 4, 9}
};

vector<vector<pair<int, int>>> graph(n);

for (auto e : edges) {
graph[e.u].push_back({e.v, e.w});
graph[e.v].push_back({e.u, e.w});
}

cout << "Minimum Spanning Tree using Kruskal's and Prim's Algorithms\n";
cout << "------------------------------------------------------------\n";

kruskal(n, edges);
prim(n, graph);

return 0;
}
```

Output

```
Minimum Spanning Tree using Kruskal's and Prim's Algorithms
------------------------------------------------------------

Kruskal's MST:
0 - 1 : 2
1 - 2 : 3
1 - 4 : 5
0 - 3 : 6
Total MST Weight = 16

Prim's MST:
0 - 1 : 2
1 - 2 : 3
1 - 4 : 5
0 - 3 : 6
```